



Repaso JS

Para React

STRING LITERAL / TEMPLATE STRING

Uso de ``

OBJETOS / ARRAY
forEach / map / find / includes
/ filter / concat

```
let nuevoArray = array1.concat(array2)  
let objetoNuevo = {...objeto1,...objeto2}
```

DESTRUCTURING / DESTRUCTURACIÓN

Coger los datos de objetos o arrays directamente y de manera múltiple, para extraerlos a variables o constantes

Destructuración objetos:

```
let persona = {  
  nombre: "redu",  
  edad: 30,  
  altura: 190  
}  
let { nombre, edad, altura } = persona;  
  
console.log(nombre);
```

DESTRUCTURING / DESTRUCTURACIÓN

Coger los datos de objetos o arrays directamente y de manera múltiple, para extraerlos a variables o constantes

Destructuración objetos:

*** PODEMOS PONER NUEVOS NOMBRES A VARIABLES COMO:
`let { nombre:n, edad:e, altura:a } = persona;`

dESTRUCTURACIÓN ARRAY

```
let meses = ["enero", "febrero", "marzo"];  
let [enero, febrero, marzo] = meses;  
console.log(enero);
```


DESTRUCTURING / DESTRUCTURACIÓN

Coger los datos de objetos o arrays directamente y de manera múltiple, para extraerlos a variables o constantes

Destructuración objetos:

Destructuración en una función:

```
function verMeses() {  
  return {  
    nombre: "redu",  
    edad: 30  
  }  
}  
  
const { nombre, edad } = verMeses();  
  
console.log(edad)
```

OPERADOR TERNARIOS

let variable = condición ? valor1 : valor2

NULLISH COALESCING OPERATOR

Operador de coalescencia nula

```
let a = 1;  
let b = 2;
```

```
let c = a == null ? 0 : a;
```

Se puede hacer con:

```
let c = a ?? 0;
```

```
console.log(c) // devuelve 1
```


OPTIONAL CHAINING

encadenamiento opcional

```
let objeto = {  
  nombre:"Redu",  
  contacto : {  
    tel:"23242342",  
    ciudad: "Granada"  
  }  
}
```

```
let ciudad = objeto?.contacto?.ciudad;  
** ahora si la ciudad no existe, devolveria undefined;
```

FUNCIONES / RETURN

(VALORES POR DEFECTO)

```
function hacerAlgo(val1=3, val2=5) {  
    return val1+val2;  
}  
hacerAlgo(null,7);
```

FUNCIONES / RETURN

(NUM,STRING,OBJETOS,ARRAY,FUNCIONES)

```
function hacerAlgo() {  
    return ....  
}  
hacerAlgo();
```

```
function hacerAlgo() {  
    return hacerOtraCosa()  
}  
hacerAlgo();
```

***** Con return se termina el código

FUNCIONES / ARROW FUNCTION

```
const fn1= () => { }  
const fn2 = param => param;  
const fn3 = (param1,param2) => { ....}
```

**** Ojo al retornar un objeto. Utilizar ();**

FUNCIONES ANÓNIMAS

```
console.log(function () {  
  return "Estoy haciendo algo";  
})();
```


USO DE EXPORT / IMPORT

*USO DE EXPORT CON ECMASCRIPT MODULE

Utilizamos import y export para comunicar módulos de javascript y se puede exportar, números, string, objetos, funciones, etc...

```
export const = 2
export palabra = "SSS"
export function add() {
  {
    en el otro archivo:
    import add from './add.js'
```

OJO::: HAY QUE PONER <script type="module" archivo.js ></script>

```
**** podemos importar varios:
import {add,multiplicar,title} from "./archivo.js";
```

```
**** EXPORTAR ALGO POR DEFECTO:
hacemos primero: export default add;
y luego importar :::: import algo from "add.js";
después utilizar algo como un alias y exportaria el que está por defecto
```

PROMESAS / ASYNC AWAIT / FETCH

ASYNC AWAIT

EJEMPLO CON : json place holder:

<https://jsonplaceholder.typicode.com/photos>